Table des matières

Cours 08 — Mettre en place l'intégration continue 1
Objectifs 2
1. Travailler par branche, et relire un modèle 3
2. L'intégration continue, et ce qu'elle n'est pas 3
3. Le workflow, ligne à ligne 4
4. La forme : le linteur 6
5. Le fond, sans les données 8
6. L'entrepôt jetable, et le ménage 9
7. Les secrets, et ce qui entre au dépôt 10
8. La fraîcheur, et ce qu'un avertissement vaut 10
9. Le contrat 12
10. Ce qu'un agent lit du `manifest.json` 12
11. Sur une autre plateforme 13
Points clés à retenir 14
Exercices 15

Cours 08 — Mettre en place l'intégration continue

Théorie · Cours 8/9 — Formation Analytics Engineering · avancé

Où vous êtes : `RAW` → `STAGING` → `INTERMEDIATE` → `MARTS` → `PRESENTATION`. Le projet tourne en régime permanent depuis le Cours 07. Ce cours ne touche à aucune couche : il met le code sous surveillance, et il règle ce qui surveille les données.

En 30 secondes :

- Un changement passe par une branche et une demande de tirage. La revue d'un modèle dbt porte sur le grain, les tests et l'étage de la logique, pas sur la syntaxe.
- L'intégration continue rejoue vos vérifications à chaque changement proposé. Elle ne fait circuler aucune donnée, et elle ne déploie rien.
- Un workflow tourne sur une machine neuve : tout ce dont il a besoin s'installe dans le fichier, à des versions exactes.

© 2026 iAlexMG. Tous droits réservés.

- La forme se vérifie par un linteur, sqlfluff. Son profil se déclare dans `.sqlfluff`, sous `[sqlfluff:templater:dbt]`, et nulle part ailleurs.
- Le fond se vérifie par `dbt build` sur un entrepôt jetable qui n'a que la structure des sources. Sur des tables vides, le test unitaire et le contrat sont les seuls tests qui voient quelque chose.
- La fraîcheur d'une source se règle sur sa cadence. Un seuil de 72 heures sur une source mensuelle est une panne d'alerte programmée.
- Un avertissement laisse la CI au vert. Ce qui doit bloquer une fusion est une erreur.
- Le `manifest.json` est ce qu'un agent lit pour savoir ce que l'entrepôt contient, et il ne dit que ce que vous y avez écrit.

L'idée du cours. Jusqu'ici, la qualité du projet reposait sur vous : vous lanciez `dbt build` avant de commiter, et vous lisiez le résultat. Ce cours la fait reposer sur une procédure. Vous passez le code au linteur, vous déclarez ce que l'étoile des ventes promet à ceux qui la lisent, vous réglez la fraîcheur des sources sur leur vraie cadence, et vous montez une vérification qui tourne toute seule sur une machine qui n'a jamais vu votre poste. Puis vous proposez un changement d'un seul caractère, que personne ne verrait en revue, et vous regardez la vérification le refuser.

Objectifs

- Relire un modèle dbt dans une demande de tirage, et nommer ce que la revue doit y chercher.
- Dire ce que l'intégration continue protège, et la distinguer du déploiement et de l'orchestration.
- Lire un workflow ligne à ligne : déclencheurs, droits, tâche, étapes, versions.
- Configurer le linteur, l'exécuter, et dire ce que sa correction automatique abîme.
- Déclarer le profil du linteur là où il se déclare, et reconnaître les deux façons de se tromper.
- Faire tourner `dbt build` sans données, et dire ce qu'il voit et ce qu'il ne voit pas.
- Monter un entrepôt jetable pour la CI, et nettoyer les schémas de travail du poste.
- Versionner le profil de la CI sans rien laisser fuir, et dire où vont les secrets sur une plateforme.
- Régler la fraîcheur d'une source sur sa cadence, et dire pourquoi elle ne va pas dans la CI.
- Dire ce qu'un avertissement fait au verdict de la CI.
- Poser un contrat sur un modèle, et dire ce qu'il attrape qu'aucun test de données n'attrape.
- Lire le `manifest.json` comme le ferait un agent, et dire ce qui reste humain.

© 2026 iAlexMG. Tous droits réservés.

1. Travailler par branche, et relire un modèle

Labo : `Labo_08_AE.md`, étape 12.

Vous travaillez seul depuis le Cours 01, en ligne droite, sur la branche principale. Ça tient tant que personne d'autre ne touche au projet et que rien ne presse. Les deux cessent vite d'être vrais.

Le geste de base est la branche : une ligne de travail nommée d'après ce qu'on y fait, où l'on commite librement sans déranger `main`. Quand le changement est prêt, on ouvre une demande de tirage (*pull request*), qui dit « voici mes changements, regardez-les avant que je les fusionne ». C'est là que la revue a lieu, et c'est là que la vérification automatique s'accroche. Les branches, la fusion et la demande de tirage appartiennent aux Cours 02 et 04 de la formation Git et GitHub ; ici, on suppose les commandes acquises, et on s'occupe de ce qui change quand le code est un projet analytique.

Ce qui change, c'est ce qu'on regarde. Sur un modèle dbt, les questions utiles sont peu nombreuses, et ce sont toujours les mêmes.

On vérifie	**Parce que**
Le grain est déclaré et un test le prouve	un grain faux est l'erreur la plus chère et la plus silencieuse
`ref()` et `source()` partout, aucun nom de table en dur	sinon le graphe, l'ordre d'exécution et le lignage mentent
La logique vit au bon étage	une règle de métier au `STAGING` est une règle que personne ne retrouvera
Le modèle a une description	une documentation remise à plus tard n'est jamais écrite
Un test couvre le changement	un changement sans test est une promesse sans preuve
Ce qui dépend du modèle	un fait lu par un tableau de bord ne change pas comme une vue de travail
Aucun secret, aucun chemin personnel	ils restent dans l'historique même après leur suppression

La sixième ligne est celle qu'on oublie, et dbt y répond sans qu'on cherche : le graphe du projet dit que `fct_ventes` sert douze nœuds, le résumé de la couche `PRESENTATION`, l'`exposure` du tableau de bord, neuf tests de données et le test unitaire. Un changement à ce modèle se relit avec cette liste sous les yeux.

Piège — la revue qui ne relit que la syntaxe. Le réflexe est de lire la requête et de conclure qu'elle a l'air bonne. Ce qui casse un entrepôt n'est presque jamais une faute de syntaxe : c'est une borne de jointure, un grain non déclaré, une règle posée au mauvais étage. Le labo vous fera proposer un changement d'un seul caractère, `<` devenu `<=`, que n'importe quel relecteur laisserait passer. La syntaxe, la machine s'en charge (section 4) ; les décisions, c'est le travail de la revue, et les bornes, celui du test unitaire (section 5).

2. L'intégration continue, et ce qu'elle n'est pas

Labo : `Labo_08_AE.md`, étapes 11 et 12.

© 2026 iAlexMG. Tous droits réservés.

L'intégration continue, qu'on abrège en CI (*continuous integration*), consiste à rejouer automatiquement un ensemble de vérifications à chaque changement proposé. Rien de plus. Elle n'invente aucun test : elle lance ceux que vous avez déjà écrits, chaque fois, y compris le vendredi à 17 h.

Ce qu'elle protège, c'est votre attention plutôt que votre code. Vous savez qu'il faut lancer `dbt build` avant de fusionner, et vous le ferez neuf fois sur dix. La dixième est celle où le test de grain aurait attrapé un doublon. La CI change une discipline en garantie, et la garantie tient même quand quelqu'un d'autre que vous propose le changement.

Trois choses voisines ne sont pas elle, et les confondre mène à de mauvais montages.

Ce n'est pas	Ce que c'est	Chez Boréal
Le déploiement continu (CD)	pousser en production ce qui a passé la CI, automatiquement	la production se reconstruit par la tâche mensuelle, quand on le décide
L'orchestration	faire circuler la donnée, à heure fixe, après le chargement	le `dbt build --target prod` que le Cours 07 a confié à l'ordonnanceur de Data Engineering
Un test des données	vérifier que ce qui est dans l'entrepôt est juste	la CI de ce cours ne voit aucune ligne de la `RAW` (section 5)

Les deux lettres CI et CD voyagent ensemble, et elles ne se valent pas. Beaucoup d'équipes mûres ont une CI stricte et un déploiement volontairement manuel, et sur un projet analytique c'est souvent le bon réglage : on veut que la production se reconstruise quand on le décide, pas à l'instant où une revue est approuvée.

Rappel — le Cours 07 a posé la distinction. L'orchestration fait tourner la donnée ; la CI garde la qualité du code. L'une s'exécute sur l'entrepôt de production, à heure fixe ; l'autre sur une machine neuve, à chaque changement. Elles ne se remplacent pas, et un projet sérieux a les deux.

Chez Boréal, la preuve se lit en trois exécutions sur GitHub, mesurées sur un dépôt d'essai. Le projet de ce cours, poussé sur `main`, passe au vert en 55 secondes. La branche qui rend la borne de fin inclusive passe au rouge, à l'étape du fond, et sa demande de tirage l'affiche. Le commit qui répare la branche la fait repasser au vert, en 50 secondes. Personne n'a lancé une seule vérification à la main.

3. Le workflow, ligne à ligne

Labo : `Labo_08_AE.md`, étapes 9, 10 et 11.

GitHub Actions exécute des scripts en réponse à des événements sur un dépôt. Le vocabulaire complet est au Cours 06 de la formation Git et GitHub ; sept mots suffisent pour lire celui de Boréal.

Le mot	Ce que c'est
workflow	un fichier YAML dans `.github/workflows/`, qui dit quoi lancer et quand

© 2026 iAlexMG. Tous droits réservés.

Le mot	Ce que c’est
déclencheur (`on:`)	l’événement qui lance le workflow
tâche (`jobs:`)	un bloc de travail, exécuté sur une seule machine
runner (`runs-on:`)	cette machine, fournie par GitHub et détruite après usage
étape (`steps:`)	une commande (`run:`) ou une action réutilisable (`uses:`), dans l’ordre
droits (`permissions:`)	ce que le jeton prêté à la tâche peut faire sur le dépôt
artefact	un fichier que la tâche dépose, et qu’on télécharge après coup

Le workflow de Boréal tient en soixante-seize lignes, dont une vingtaine de commentaires. Sa tête dit quand il tourne et ce qu’il a le droit de faire :

```yaml
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

permissions:
  contents: read
```

Deux déclencheurs, deux questions. Une demande de tirage vers `main` demande si un changement peut entrer ; une poussée sur `main` demande si ce qui est entré tient toujours. Et le jeton que GitHub prête à la tâche ne peut que lire le dépôt : une vérification n’a rien à écrire.

Le point qui explique la longueur du fichier est celui qu’on oublie : le runner est neuf à chaque exécution. Chez Boréal, un Ubuntu 24.04 avec Python 3.11.16, mesuré. Il ne connaît ni votre environnement conda, ni dbt, ni votre `profiles.yml`, ni votre entrepôt. Tout ce dont la tâche a besoin s’installe ou se fabrique dans le fichier, et c’est ce qui garantit que la vérification marchera chez le prochain qui clone le dépôt.

La tâche porte neuf étapes, dans un ordre qui n’est pas arbitraire.

L’étape	Ce qu’elle fait	Sa durée mesurée
Récupérer le dépôt	`actions/checkout`	1 s
Installer Python 3.11	`actions/setup-python`, avec un cache des paquets	1 s
Installer la pile	`pip install -r ci/requirements.txt`	20 s

© 2026 iAlexMG. Tous droits réservés.

L'étape	Ce qu'elle fait	Sa durée mesurée
Installer `dbt_utils`	`dbt deps`	3 s
La forme	`sqlfluff lint models tests snapshots`	10 s
Monter la structure des sources	une ligne de Python, section 5	1 s
Le fond	`dbt build --target ci --profiles-dir ci`	6 s
La documentation	`dbt docs generate`	4 s
La publier	`actions/upload-artifact`	1 s

`dbt deps` passe avant le linteur, et ce n'est pas de la politesse. Le linteur compile le projet, et le projet appelle `dbt_utils` : sans le paquet installé, il s'arrête sur `UninstalledPackagesFoundError`, avant d'avoir lu une ligne. L'ordre se prouve en l'inversant, et le labo le fera.

La pile s'installe à des versions exactes, `dbt-duckdb==1.11.0` et non `dbt-duckdb>=1.11,<1.12`, et ce sont celles du poste. Une fourchette ferait installer au runner la dernière version du jour, et la CI passerait du vert au rouge un matin sans que personne ait touché au code. Le `requirements.txt` de la formation garde ses fourchettes, parce qu'il installe un poste ; celui de la CI épingle, parce qu'il fabrique une preuve.

La dernière étape dépose un artefact, `documentation-dbt`, de 654 507 octets mesurés : le site de la documentation, le `manifest.json` et le `catalog.json`. La documentation cesse ainsi de vivre sur votre seul poste, et la section 10 dira qui la lit.

À retenir — un workflow se prouve en le faisant tourner, pas en le relisant. Une version antérieure de ce cours passait à sqlfluff une option qui n'existe pas, et la faute a traversé toutes les relectures, parce que le fichier se lisait bien et que personne ne l'avait lancé. La règle vaut pour les supports comme pour vos projets : un workflow enseigné s'exécute pour de vrai avant d'entrer dans un document, comme un chiffre.

4. La forme : le linteur

Labo : `Labo_08_AE.md`, étapes 2, 3 et 4.

Un linteur lit du code sans l'exécuter et signale ce qui s'écarte d'une convention : l'indentation, la casse, les références ambiguës, l'ordre des clauses. Celui du SQL s'appelle sqlfluff. Il ne touche pas à l'entrepôt, il tourne en une quinzaine de secondes, et il met fin aux débats de style en revue : la convention est écrite une fois, dans un fichier, et la machine la fait respecter.

Un modèle dbt n'est pas du SQL : c'est de la Jinja qui en produit. sqlfluff le lit donc en deux temps. Son templater dbt compile d'abord le projet, comme le ferait `dbt compile`, puis le linteur lit le SQL qui en sort. Compiler demande un profil et une cible, et c'est là que tout se joue.

Le profil du linteur se déclare dans `.sqlfluff`, sous la section `[sqlfluff:templater:dbt]`, par `profiles_dir` et `target`. Nulle part ailleurs. sqlfluff n'a pas d'option `--profiles-dir` en ligne de commande : mesuré, `sqlfluff lint --profiles-dir ci` rend `Error: No such option '--profiles-dir'.` et le code 2. Et une clé rangée sous une autre section est ignorée sans un mot, ce qui donne deux façons de se tromper, dont une sournoise.

© 2026 iAlexMG. Tous droits réservés.

Ce qui est mal rangé	Sur le poste	Sur le runner
`profiles_dir` seul, sous `[sqlfluff]`	« The profile 'boreal_ae' does not have a target named 'ci' »	« Could not find profile named 'boreal_ae' »
`profiles_dir` et `target`, sous `[sqlfluff]`	rien : le linteur passe, en lisant votre profil personnel et sa cible `dev`	« Could not find profile named 'boreal_ae' »

La seconde ligne est celle qui coûte une matinée. Sur votre poste, tout est vert, parce que le templater retombe sur `~/.dbt/profiles.yml` ; sur la machine neuve, qui n'a pas ce fichier, tout casse, et le message parle d'un profil introuvable alors que le fichier du profil est dans le dépôt. Un chemin en `_dir` se résout depuis le `.sqlfluff`, pas depuis le dossier où l'on lance la commande : `profiles_dir = ci` désigne donc le dossier `ci/` du dépôt, où qu'on soit.

Les règles de sqlfluff portent un code à quatre caractères, dont les deux lettres disent la famille : `LT` pour la mise en page, `RF` pour les références, `ST` pour la structure, `CP` pour la casse. Sur le projet tel que le Cours 07 le laisse, et avec le réglage du labo, le linteur rend 46 écarts dans 13 fichiers : 38 d'indentation (`LT02`), 3 références non qualifiées (`RF02`), 2 références qu'il croit introuvables (`RF01`), 2 colonnes sur la ligne du `select` (`LT09`) et un `where` qui ne commence pas sa ligne (`LT14`).

Deux règles sont écartées par le fichier, et chacune a sa raison, écrite à côté. `ST06` veut les colonnes simples avant les calculs, et range donc les colonnes par forme d'expression plutôt que par sens : dans `stg_commandes`, sa correction pousse les deux dates après `lot_source`, la colonne technique qui fermait la liste. L'ordre des colonnes d'un modèle est une décision de conception, la clé d'abord et les colonnes techniques à la fin, et la règle la défait dans dix modèles de `STAGING` et deux tests. `ST09` veut la table nommée en premier à gauche d'une condition de jointure, et retournerait quinze jointures sans rien corriger. Sans ces deux exclusions, le linteur rend 73 écarts dans 30 fichiers. Écarter une règle n'est pas une faiblesse ; l'écarter sans écrire pourquoi en est une.

`sqlfluff fix` corrige ce qu'il sait corriger : 41 écarts dans 12 fichiers, en 16 secondes. Il en laisse 5. Et il ne relit pas ce qu'il écrit, ce que deux mesures établissent.

La première : la liste des noms de mois de `dim_date`, que le Cours 04 avait écrite sur trois lignes, sort du correcteur ainsi.

```
    [
        'janvier', 'février', 'mars', 'avril', 'mai', 'juin',
        'juillet', 'août', 'septembre', 'octobre', 'novembre',
        'décembre'
    ][month(date_day)
    ] as nom_mois,
```

C'est conforme à la règle et illisible. La correction se fait à la main, la liste sur ses propres lignes et l'indice collé au crochet.

La seconde tient à la Jinja. Le correcteur ne voit que la branche d'un `{% if %}` que la compilation a prise. Sans le réglage `template_blocks_indent = false`, il trouve seize écarts dans le seul snapshot, en corrige quinze, indente tout son corps et laisse la branche non prise collée à la marge : le fichier sort à moitié décalé, et il reste juste, ce qui est pire.

© 2026 iAlexMG. Tous droits réservés.

Restent les cinq écarts que `fix` ne sait pas corriger, et ils ne sont pas cosmétiques. Trois sont des références non qualifiées dans une sous-requête. Une colonne sans préfixe, à l'intérieur d'une sous-requête, se cherche d'abord dans la table de la sous-requête, puis dans la requête qui l'entoure, sans prévenir. Mesuré sur l'entrepôt de Boréal :

```
select count(*) from marts.fct_ventes
where cle_produit in (select cle_produit from marts.dim_date);
```

`dim_date` n'a pas de colonne `cle_produit`. La requête ne se plaint pas : elle prend celle du fait, compare chaque ligne à elle-même, et rend 80 618, toutes les lignes. Un préfixe, `d.cle_produit`, l'aurait fait échouer sur le champ. C'est ce que `RF02` demande, et c'est pour cela qu'il ne se désactive pas.

Les deux autres sont de faux positifs. `RF01` prend pour une table le nom `etape`, qui est une colonne de type structure dépliée par un `unnest` au Cours 06. Un commentaire `-- noqa: RF01` en bout de ligne dit au linteur de ne pas signaler cette ligne-là, et à celui qui la relit qu'on a regardé.

Piège — `fix` n'est pas une revue. Il a corrigé 41 écarts en seize secondes, et rendu un modèle illisible au passage. Après un `fix`, on relit le `git diff` en entier avant de commiter, comme on relirait le travail d'un collègue pressé. Le correcteur écrit aussi ses fichiers en fins de ligne Unix : Git le signale sous Windows et normalise au commit, sans effet sur le contenu.

5. Le fond, sans les données

Labo : `Labo_08_AE.md`, étapes 7, 8, 10 et 12.

La forme ne dit rien du fond. Le second geste de la CI est de construire le projet, `dbt build`, et de faire tourner ses tests. Il faut pour cela un entrepôt, et c'est ici que le montage de Boréal fait un choix qu'il faut comprendre.

Le runner n'a pas les données. L'entrepôt pèse 92 Mo sur le poste, sa `RAW` appartient à Data Engineering, et ses données ne doivent pas partir sur une machine publique. Ce qu'il peut avoir, c'est la structure des sources : les deux schémas `raw` et `argent`, et leurs 25 tables, sans une ligne. Une requête DuckDB la tire de l'entrepôt en un fichier de 27 lignes, `ci/structure_sources.sql`, qui se versionne, et une ligne de Python la monte dans un fichier DuckDB neuf, en un cinquième de seconde.

Sur cette structure vide, `dbt build` fait tout ce qu'il fait d'habitude : 130 nœuds, 129 au vert et l'`exposure`, qui n'a rien à exécuter. Il bâtit 30 objets, et trois seulement ont des lignes : le seed des anomalies et sa dimension, 5 lignes chacun, et `dim_date`, 1 551 jours, qui ne doit rien à la `RAW`. Tout le reste est vide.

Ce que cette construction voit, et ce qu'elle ne voit pas, se dit en un tableau, mesuré.

Le défaut proposé	**Ce que la CI en dit**
Une Jinja qui ne compile pas, un `ref()` vers un modèle absent	échec à la compilation, avant toute exécution
Une colonne mal nommée, `l.valeur_net` pour `l.valeur_nette`	`Binder Error`, `PASS=116 ERROR=1 SKIP=13`
Le type d'une colonne changé sous un contrat	échec du contrat, section 9

© 2026 iAlexMG. Tous droits réservés.

Le défaut proposé	Ce que la CI en dit
La borne de fin de la jointure temporelle rendue inclusive	échec du test unitaire, `PASS=116 ERROR=1 SKIP=13`
Un doublon, une valeur hors liste, une réconciliation fausse	rien : les 98 tests de données passent sur des tables vides

La dernière ligne est la leçon du cours. Un test de données est une requête qui ne doit rien rendre (Cours 05), et une requête sur une table vide ne rend rien : les 98 tests passent au vert sans rien vérifier. La même borne inclusive, construite sur les vraies données du poste, fait tomber le grain sur 16 couples et la réconciliation sur 4 commandes, 80 634 lignes pour 80 618 couples et 14 567,58 $ de trop ; sur la structure vide, ces deux tests ne voient rien.

Deux tests voient quelque chose sans données, et ce sont donc les tests de la CI. Le test unitaire apporte ses propres lignes : les trois ventes de son `given` éprouvent la borne de fin, et une ligne `C2` de trop suffit à le faire tomber. Le contrat, lui, ne regarde que les noms et les types, qui existent sur une table vide. Le Cours 05 a écrit le test unitaire pour la logique la plus fragile du projet ; ce cours lui trouve sa vraie place.

Pourquoi — une CI sans données plutôt qu'une CI sur un échantillon. Un échantillon de la `RAW` versionné dans le dépôt rendrait les tests de données utiles, et poserait trois problèmes : quelqu'un doit le choisir et le tenir à jour, il fait entrer des données de l'entreprise dans un dépôt, et un test qui passe sur l'échantillon ne dit rien de la production. La structure vide ne prétend rien. Ce qu'elle vérifie, le code, elle le vérifie entièrement ; les données se vérifient là où elles sont, par `dbt build` sur l'entrepôt de production, que la tâche du Cours 07 lance chaque mois. Et même là, dbt construit un modèle avant de le tester : un test en échec arrête la suite de la chaîne, il ne défait pas le modèle fautif, qui reste écrit jusqu'à la correction.

Une dernière conséquence : la structure date du jour où on l'a tirée. Le jour où Data Engineering ajoute une colonne à une table de la `RAW` et qu'un modèle la lit, la CI tombera sur un `Binder Error`, et c'est le signal qu'il faut regénérer le fichier. Une CI qui casse pour une raison qu'on comprend n'est pas une CI fragile.

6. L'entrepôt jetable, et le ménage

Labo : `Labo_08_AE.md`, étapes 1, 8 et 13.

La CI construit dans une cible à elle, `ci`, déclarée dans un profil versionné, `ci/profiles.yml`, qui pointe un fichier DuckDB nommé `ci.duckdb`. Le chemin est relatif, et c'est l'exception à la règle du Cours 03, qui voulait un chemin absolu : la CI se lance toujours depuis la racine du dépôt, et un chemin absolu désignerait un dossier de votre poste que le runner n'a pas.

La cible `ci` n'est pas `prod`, et la macro `generate_schema_name` du Cours 07 lui donne donc les noms préfixés : les modèles de la CI atterrissent dans `dev_staging`, `dev_marts` et `dev_presentation` de l'entrepôt jetable. Le projet connaît maintenant trois environnements, sans qu'une ligne de modèle le sache.

La cible	Le fichier	Les schémas
`dev`	l'entrepôt du poste	`dev_staging`, `dev_marts`, `dev_presentation`
`prod`	l'entrepôt du poste	`staging`, `marts`, `presentation`
`ci`	`ci.duckdb`, créé sur le runner	les mêmes que `dev`, dans un fichier qui disparaît

© 2026 iAlexMG. Tous droits réservés.

Sur le runner, le ménage se fait tout seul : la machine est détruite avec son fichier. Le snapshot n'y est donc pas exclu de `dbt build`, contrairement au poste. Sur un entrepôt neuf et vide, il n'a aucune histoire à abîmer, et `dim_concessionnaire` a besoin que sa table existe.

Sur le poste, c'est l'inverse : les schémas `dev_*` s'accumulent depuis le Cours 07. Un modèle renommé y laisse son ancienne vue, et un chiffre lu dans `dev_marts` peut dater de trois essais. Le Cours 07 l'avait annoncé : ils se jettent par une macro, lancée par `dbt run-operation`, la commande qui exécute une macro en dehors de toute construction.

La macro de Boréal, `nettoyer_dev`, tient en douze lignes, et deux décisions la rendent sûre. Elle ne nomme aucun schéma en dur : elle les demande à `generate_schema_name`, la macro qui les a fabriqués, et suit donc le jour où la convention change. Et elle commence par un verrou : sur la cible `prod`, elle refuse et rend une erreur avant de toucher à quoi que ce soit. Mesuré, elle jette les trois schémas `dev_*` d'un coup ; sur `--target prod`, elle rend le code 1 et ne touche à rien. Le schéma `snapshots`, partagé entre les cibles, n'est pas dans sa liste : son histoire ne se reconstruit pas par `dbt build`.

7. Les secrets, et ce qui entre au dépôt

Labo : `Labo_08_AE.md`, étape 1.

La CI de Boréal n'a aucun secret, et c'est un luxe qu'il faut savoir nommer. Son entrepôt est un fichier créé sur place, sans compte ni mot de passe ; rien, dans le workflow, ne vaut la peine d'être volé. Sur Snowflake ou Databricks, ce ne sera plus vrai, et la section 11 dit ce qui change.

Le profil de la CI se versionne, et cela semble contredire le Cours 03, qui mettait le `profiles.yml` hors du dépôt. Ce n'est pas le même fichier. Celui de votre poste porte un chemin de votre machine, et porterait un compte sur une plateforme ; celui de la CI ne porte rien qu'un nom de fichier relatif, et sur une plateforme il ne porterait que des appels `env_var(...)`, c'est-à-dire des noms de variables. Il dit où chercher les valeurs, il ne les contient pas.

Reste à le faire entrer, et le `.gitignore` du Cours 01 s'y oppose. Sa ligne `profiles.yml` vise tout fichier de ce nom, où qu'il soit, `ci/profiles.yml` compris : mesuré, `git add` le refuse, et `git check-ignore -v` nomme la ligne coupable. La parade est une exception, `!ci/profiles.yml`, posée après la règle qu'elle contredit. Dans le même geste, deux lignes pour l'entrepôt jetable, `ci.duckdb` et son journal `ci.duckdb.wal` : le linteur crée ce fichier à la racine du dépôt dès sa première compilation, et rien ne l'aurait empêché d'entrer au premier `git add .`.

Piège — le secret qui reste dans l'historique. Un mot de passe commité puis supprimé au commit suivant est toujours là : Git garde tout. Le retirer demande de réécrire l'historique, et sur un dépôt déjà poussé il faut de toute façon tenir le secret pour brûlé et le changer. La seule règle qui tienne : vérifier avant le premier `git push`, jamais après.

8. La fraîcheur, et ce qu'un avertissement vaut

Labo : `Labo_08_AE.md`, étape 5.

Les tests répondent à « la donnée est-elle conforme ». Ils ne répondent pas à « la donnée est-elle récente » : un `not_null` passe parfaitement sur une table qui n'a pas bougé depuis trois semaines. C'est le rôle de `dbt source freshness`.

© 2026 iAlexMG. Tous droits réservés.

La commande lit, pour chaque source, la plus récente valeur d'une colonne d'horodatage de chargement, la compare à l'heure qu'il est, et rend un verdict selon deux seuils : `warn_after`, au-delà duquel elle avertit, et `error_after`, au-delà duquel elle échoue. Chez Boréal, la colonne est `_charge_le`, que Data Engineering pose sur chaque ligne de la `RAW`. Les deux réglages se rangent sous `config:`, et dbt 1.12 les lit ainsi sans avertissement de dépréciation.

```yaml
sources:
  - name: raw
    config:
      loaded_at_field: _charge_le
      freshness:
        warn_after: {count: 35, period: day}
        error_after: {count: 40, period: day}
```

Toute la difficulté est dans les deux nombres, et ils ne se choisissent pas ronds. Ils se choisissent sur la cadence de la source. L'ERP et les garanties arrivent une fois par mois, la nuit qui suit la fin du mois : l'âge normal de leur dernier chargement va de zéro à trente et un jours, d'où 35 jours pour l'avertissement et 40 pour l'erreur. La télémétrie arrive chaque nuit : 36 heures et 72 heures, réglées sur sa table. Le registre des actifs de l'usine et ses étiquettes sont des exports complets, livrés une fois : leur âge ne dit rien, et `freshness: null` le déclare. Le schéma `argent` n'a pas de colonne de chargement ; sa fraîcheur se surveille chez Data Engineering, à la sortie de son pipeline.

Quatre scénarios, mesurés sur l'entrepôt de Boréal et sur une copie vieillie à la main.

L'état des sources	Le verdict	Le code de sortie
Tout chargé depuis quatre heures et demie	10 `PASS`	0
ERP à 20 jours, réclamations à 38, télémétrie à 4	8 `PASS`, 1 `WARN`, 1 `ERROR STALE`	1
La même copie, seuils de 36 et 72 heures partout	10 `ERROR STALE`	1
Réclamations à 38 jours, le reste à jour	9 `PASS`, 1 `WARN`	0

La deuxième ligne est un mois ordinaire, vu au 20 : les sources mensuelles sont vertes, les réclamations ont manqué leur livraison et avertissent, la télémétrie a cessé d'arriver et échoue. C'est exactement ce qu'on veut savoir. La troisième ligne est la même situation sous des seuils ronds, 36 et 72 heures pour tout le monde : dix erreurs, dont huit sur des sources parfaitement à jour. Sur une source mensuelle, un seuil de 72 heures la met en erreur du troisième jour à la fin du mois, vingt-sept jours sur trente, en fonctionnement normal. C'est l'usure d'alerte que le Cours 05 dénonçait : une équipe qui voit rouge tous les jours cesse de regarder, et le jour où la télémétrie tombe vraiment, personne ne le voit.

À retenir — un seuil de fraîcheur se règle sur la cadence de la source, jamais sur une valeur ronde. Il s'écrit avec sa raison à côté, dans le YAML, pour que celui qui le lira dans un an sache s'il doit changer quand la cadence change.

La quatrième ligne dit ce qu'un avertissement vaut pour une machine : rien. Avec un seul `WARN`, la commande rend 0, et une étape de CI qui rend 0 passe au vert. Il en va de même d'un test réglé en `severity: warn` au Cours 05, qui rendait `WARN 633` et le code 0. La règle en découle : ce qui doit bloquer une fusion ou arrêter une tâche est une erreur ; un avertissement s'adresse à un humain qui lit le journal, et il ne sert que si quelqu'un le lit.

© 2026 iAlexMG. Tous droits réservés.

La fraîcheur, enfin, ne va pas dans la CI. Sur l'entrepôt vide du runner, les dix sources sont périmées par construction, mesuré, 10 `ERROR STALE` et le code 1 : l'étape serait rouge à chaque exécution. La fraîcheur mesure les données, pas le code. Sa place est là où les données arrivent : dans la tâche planifiée du Cours 07, juste avant `dbt build`, là où une source périmée doit arrêter la chaîne plutôt que publier des chiffres d'hier sous la date d'aujourd'hui.

9. Le contrat

Labo : `Labo_08_AE.md`, étapes 6 et 8.

Un modèle que d'autres lisent a une interface : ses colonnes, leurs noms et leurs types. Business Intelligence lit `fct_ventes` ; l'`exposure` du Cours 05 le déclare. Si une retouche change le type d'une colonne, le tableau de bord le découvre le lendemain, et les tests de données ne le voient pas, puisqu'ils regardent des valeurs et non des types.

Un contrat de modèle déclare cette interface, et dbt la fait respecter. On l'active par `contract: enforced: true` dans la configuration du modèle, et on donne à chaque colonne son `data_type`. Avant de créer la table, dbt compare les colonnes que la requête produit à celles du contrat, noms, types et nombre, et refuse de construire si elles diffèrent.

La mesure est nette. Sur la cible `ci`, une retouche qui écrit `l.quantite::integer` à la place de `l.quantite` fait tomber le contrat, avant qu'une seule ligne soit écrite, avec un tableau qui dit tout.

```
| column_name | definition_type | contract_type | mismatch_reason    |
| ----------- | --------------- | ------------- | ------------------ |
| quantite    | INTEGER         | DECIMAL(18,3) | data type mismatch |
```

Sans le contrat, la même retouche passe au vert, et la colonne devient `INTEGER` en silence. Aucun test de données ne bronche, et pour une bonne raison : les 80 618 quantités de Boréal sont toutes entières, donc aucune valeur ne change. Ce qui change, c'est la promesse, et le jour où une pièce se vendra au mètre, la quantité sera tronquée sans que rien ne l'ait annoncé.

Deux limites, pour ne pas lui demander plus qu'il ne donne. Le contrat vérifie la forme de la sortie, jamais son contenu : un `fct_ventes` aux bons types et aux mauvais montants le respecte. Et il se pose sur les modèles que quelqu'un d'autre lit, pas partout : chez Boréal, sur le fait des ventes, celui que l'`exposure` déclare. Sur une vue de travail, un contrat ne ferait que ralentir chaque retouche.

10. Ce qu'un agent lit du `manifest.json`

Labo : `Labo_08_AE.md`, étape 14.

Le Cours 05 a présenté le `manifest.json` comme le seul endroit où le lignage existe sous une forme qu'un programme peut lire. La CI le publie maintenant à chaque exécution, et c'est ce qui en fait un outil pour un agent d'intelligence artificielle : un programme qui reçoit une question, choisit des outils pour y répondre, et les appelle.

© 2026 iAlexMG. Tous droits réservés.

Ce qu'il y trouve se mesure. Le manifeste du projet pèse 1 245 249 octets, et une vingtaine de lignes de Python en tirent l'essentiel : 31 modèles, 98 tests, un snapshot, un seed, 14 sources et une `exposure`. Pour `fct_ventes`, sa description, son contrat et le type de ses neuf colonnes, les deux nœuds qu'il lit, et les douze qui en dépendent. Un agent qui doit répondre « quelle est la valeur vendue par palier en 2025 » n'a pas à deviner la structure de l'entrepôt : elle lui est servie, avec le grain, les clés et les tests qui garantissent chaque colonne.

Trois usages tiennent debout aujourd'hui, et chacun laisse une part à l'humain.

L'usage	Ce que l'agent fait	Ce qui reste humain
Documenter	proposer les descriptions manquantes, à partir du SQL et des noms	valider : une description inventée est pire qu'absente
Relire	signaler dans une demande de tirage un `ref()` manquant, un modèle sans test	décider si le modèle est le bon
Répondre	traduire une question en requête sur les `MARTS`, en suivant le lignage	vérifier le chiffre avant de le diffuser

Et une limite qu'il faut savoir énoncer : le manifeste ne dit que ce que vous y avez écrit. Sur un projet sans descriptions, sans grain déclaré ni tests, un agent produit des réponses plausibles et fausses, avec l'aplomb en plus. L'ordre n'est donc pas « ajouter de l'IA au projet », mais « rendre le projet lisible, ce qui le rend exploitable par un agent ». Les sept cours précédents étaient ce travail-là.

Bâtir l'agent appartient à la formation AI Engineering : les appels d'outils à son Cours 04, l'évaluation des réponses à son Cours 05. Ici, on s'arrête à ce que le projet lui fournit, et à la raison pour laquelle il le fournit bien.

11. Sur une autre plateforme

Tout ce cours tourne sans compte et sans secret, et c'est ce qui change le plus sur Snowflake ou Databricks. L'annexe du labo donne le profil et le workflow des deux, écrits d'après la documentation ; trois différences suffisent à les lire.

La CI peut lire la vraie `RAW`. Sur une plateforme partagée, un rôle en lecture seule donne à la CI accès aux données de production, et les tests de données y retrouvent leur sens. Le prix est double : le temps d'entrepôt se paie à chaque demande de tirage, et la CI a maintenant besoin d'un secret.

Le secret vit dans les secrets du dépôt GitHub, passé à l'étape par une variable d'environnement et lu par `env_var(...)` dans le profil. GitHub le masque dans les journaux, et ne le transmet pas à un workflow déclenché depuis une copie du dépôt (*fork*), d'après sa documentation consultée le 2026-09-23. Sur Snowflake, ce secret n'est plus un mot de passe : la documentation de dbt annonce la fin de l'authentification par mot de passe au 31 août 2026, au profit de la paire de clés.

La construction coûte, et dbt a une réponse qu'on nomme ici sans l'installer : la CI mince (*slim CI*), qui compare le projet à l'état de la dernière exécution de production et ne reconstruit que ce qui a changé et ce qui en dépend, par `dbt build --select state:modified+ --state`. Elle demande deux choses que le projet de Boréal n'a pas encore : le `manifest.json` de la production, conservé d'une exécution à l'autre, et un entrepôt où lire les modèles qu'on ne reconstruit

© 2026 iAlexMG. Tous droits réservés.

pas, ce que fait l'option `--defer`. Sans le second, mesuré, un `dbt build --select fct_ventes` sur l'entrepôt jetable s'arrête dès le test unitaire, qui ne trouve pas la table de `dim_concessionnaire`.

Recoupements. La qualité à l'entrée de la chaîne appartient au Cours 11 de Data Engineering, qui pose ses contrats sur ce qui arrive dans la `RAW` ; la qualité en aval, le contrat de `fct_ventes` et les tests de l'étoile, est ici. Les deux formations se passent la main à la `RAW`, et une source qui casse doit tomber chez Data Engineering avant de tomber chez vous. Le `manifest.json` est ce qu'un agent de la formation AI Engineering lit pour savoir ce qui existe, à son Cours 04 ; la formation Architecture et Gouvernance y lit le lignage à son Cours 04, et fait du contrat un engagement entre équipes, avec un propriétaire et un recours, à son Cours 05.

Points clés à retenir

- Un changement passe par une branche et une demande de tirage. La revue d'un modèle porte sur le grain, les références, l'étage de la logique, la description, les tests et ce qui dépend du modèle ; la syntaxe, la machine s'en charge.
- La CI rejoue vos vérifications à chaque changement proposé. Ce n'est ni le déploiement, qui pousse en production, ni l'orchestration, qui fait circuler la donnée.
- Un workflow tourne sur une machine neuve : tout s'y installe, à des versions exactes, et `dbt deps` passe avant le linteur.
- Un workflow se prouve en le faisant tourner. Un fichier qui se lit bien n'est pas un fichier qui marche.
- Le profil du linteur se déclare dans `.sqlfluff`, sous `[sqlfluff:templater:dbt]`. sqlfluff n'a pas d'option `--profiles-dir`, et une clé mal rangée est ignorée sans un mot, parfois au vert sur le poste et au rouge sur le runner.
- `sqlfluff fix` corrige vite et ne relit pas : 41 écarts corrigés, un modèle rendu illisible. Une règle s'écarte avec sa raison écrite ; `RF02` ne s'écarte pas.
- La CI construit sur la structure vide des sources. Sur des tables vides, les tests de données passent sans rien vérifier ; le test unitaire et le contrat sont les tests de la CI.
- Trois environnements, `dev`, `prod` et `ci`, sans qu'un modèle le sache. Les schémas de travail du poste se jettent par une macro verrouillée sur la production.
- Le profil de la CI se versionne parce qu'il ne porte aucune valeur. Sur une plateforme, les secrets vivent dans les secrets du dépôt.
- Un seuil de fraîcheur se règle sur la cadence de la source : 35 et 40 jours sur une source mensuelle, 36 et 72 heures sur une source quotidienne. La fraîcheur ne va pas dans la CI.
- Un avertissement rend 0 et laisse la CI au vert. Ce qui doit bloquer est une erreur.
- Un contrat fige les noms et les types d'un modèle que d'autres lisent. Il attrape un changement de type que les données ne montrent pas, et ne vérifie aucun contenu.

© 2026 iAlexMG. Tous droits réservés.

- Le `manifest.json` est ce qu'un agent lit du projet, et il ne dit que ce que vous y avez écrit.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_08_AE.md`. Faites-les après le labo : plusieurs énoncés supposent que la CI a tourné au moins une fois.

© 2026 iAlexMG. Tous droits réservés.